

Software Architecture 2024[2025]

Lecture 4, part 3/3

Architectural views

Wouter Joosen

Laurens Sion

intangible

SOFTWARE ARCHITECTURE

Recap from
“architecture
defined” (lecture
03)

Structure: elements, and (de)composition

Principles: tactics, patterns, styles, design principles

Rationale: decisions, assumptions

=> In function of **Software Quality**

shapes

(SOFTWARE-INTENSIVE) SYSTEM

tangible

*conveys
information
about*

tangible

**Architecture
description**

Recap from “architecture defined”

1. Systems can be described from **many stand points**
 2. ..in most cases, systems **have to be** described from many stand points
 - » Different non-quality requirements affect different aspects of the system
-
- › No single structure can claim to be “**the**” architecture
 - » But **many structures will convey** architectural information
 - » Together, these structures **comprise the architecture description**

Recap from “architecture defined”

1. Systems can be described from **many stand points**
 2. ..in most cases, systems **have to be** described from many stand points
 - » Different non-quality requirements affect different aspects of the system
- › No single structure can claim to be “**the**” architecture
- » But **many structures will convey architectural information**
 - » Together, these structures **comprise the architecture description**



OUR FOCUS TODAY

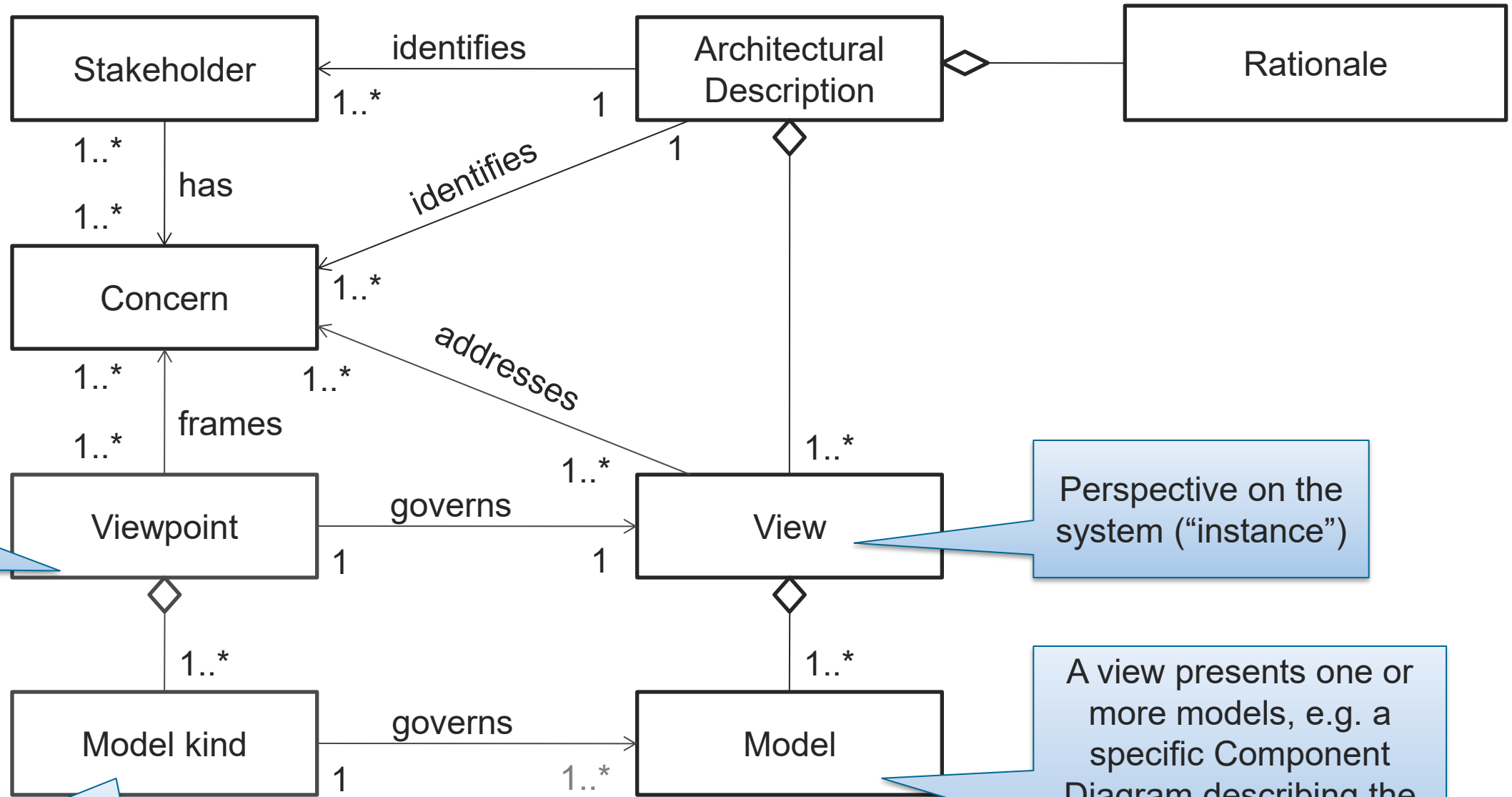
Outline of this lecture

1. Terminology and concepts
2. Architectural frameworks
3. Viewpoints → [2026] *we start from this point and revisit broader concepts and terminology later*
4. Model kinds → [2026] *what we typically produce in this course*

1. Terminology and concepts

ISO 42010 System and software engineering – Architecture description
(2000/2011)





“viewtype”

Perspective on the system (“instance”)

e.g. UML specification for Component Diagram (syntax)

A view presents one or more models, e.g. a specific Component Diagram describing the system

ISO 42010

Architectural view

- › An architectural view is a simplified description (**abstraction**) of a specific system
 - › From a particular **perspective** (stakeholders and concerns)
 - › Omitting entities that are not relevant to this perspective
- › Contains **models** addressing the *concerns*
 - › Is of a certain **model kind**
- › Is the result of applying a **viewpoint** (and refers to it)
 - › a view *adheres* to a viewpoint (“*is structured in accordance to*”/ “*is governed by*”)

London tube map



What is the **main purpose** of this model?

Can we use it for **calculating distances**?

London tube map

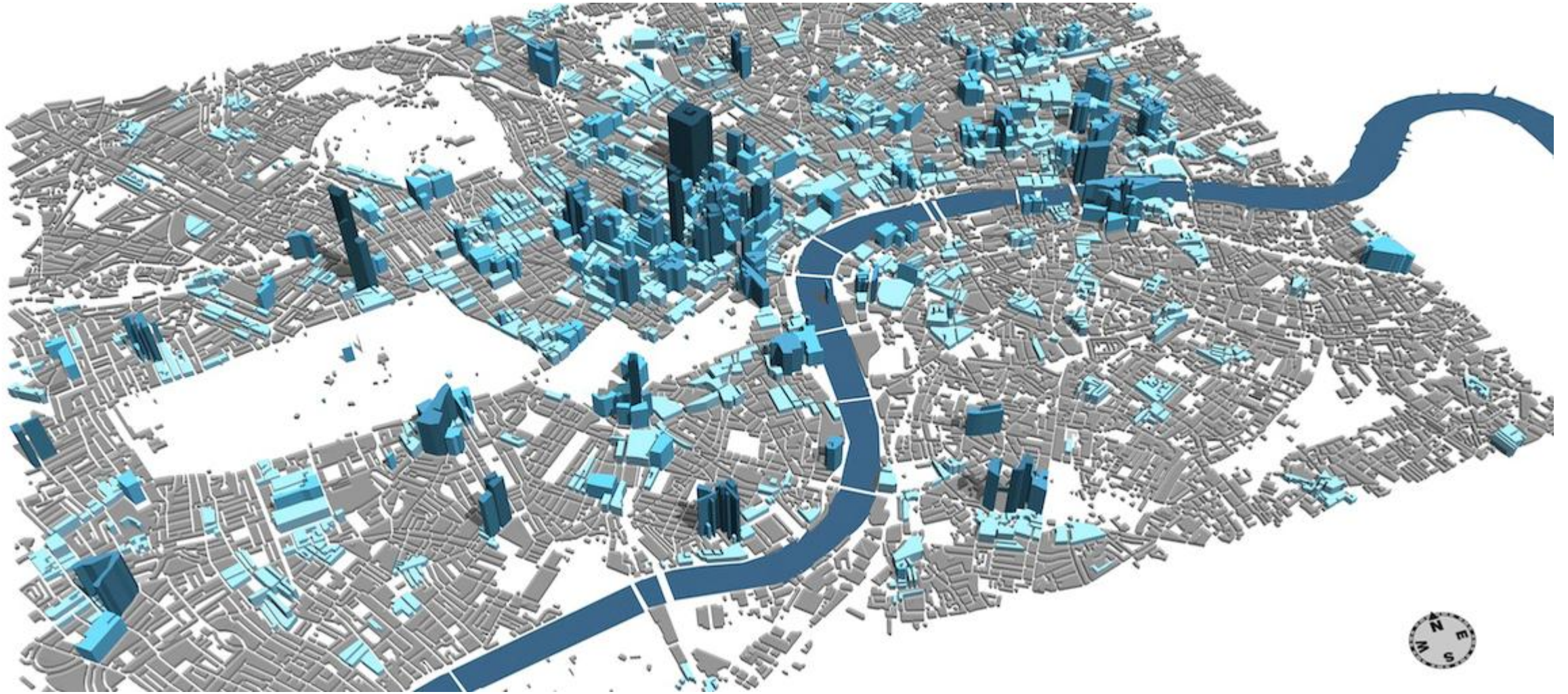


From "Patterns, Principles, and Practices of
Domain-Driven Design"
By Scott Millett, Nick Tune 2/26/2026

'Reality'

iMinds-DistriNet

London map



Viewpoint

- › A **view** (instance level) is structured in accordance to a **viewpoint** (class/type level)
- › An **viewpoint** is characterized by:
 - ›› a set of **concerns** to be addressed
 - ›› a set of **stakeholders** interested in how they are addressed
- › **Viewpoint** dictates the **conventions** for constructing, interpreting, using and analyzing the view
 - ›› **Model kinds** (languages, notations) -> semantics
 - ›› Modeling technique, patterns, design guidelines and heuristics/tools to aid with creation of the view
 - ›› **Correspondence** rules linking the models across and within viewpoints
 - ›› Criteria and methods for checking consistency and completeness
 - ›› Evaluation or analysis methods

In this course...

- › Terminology aligned to **ISO 42010**

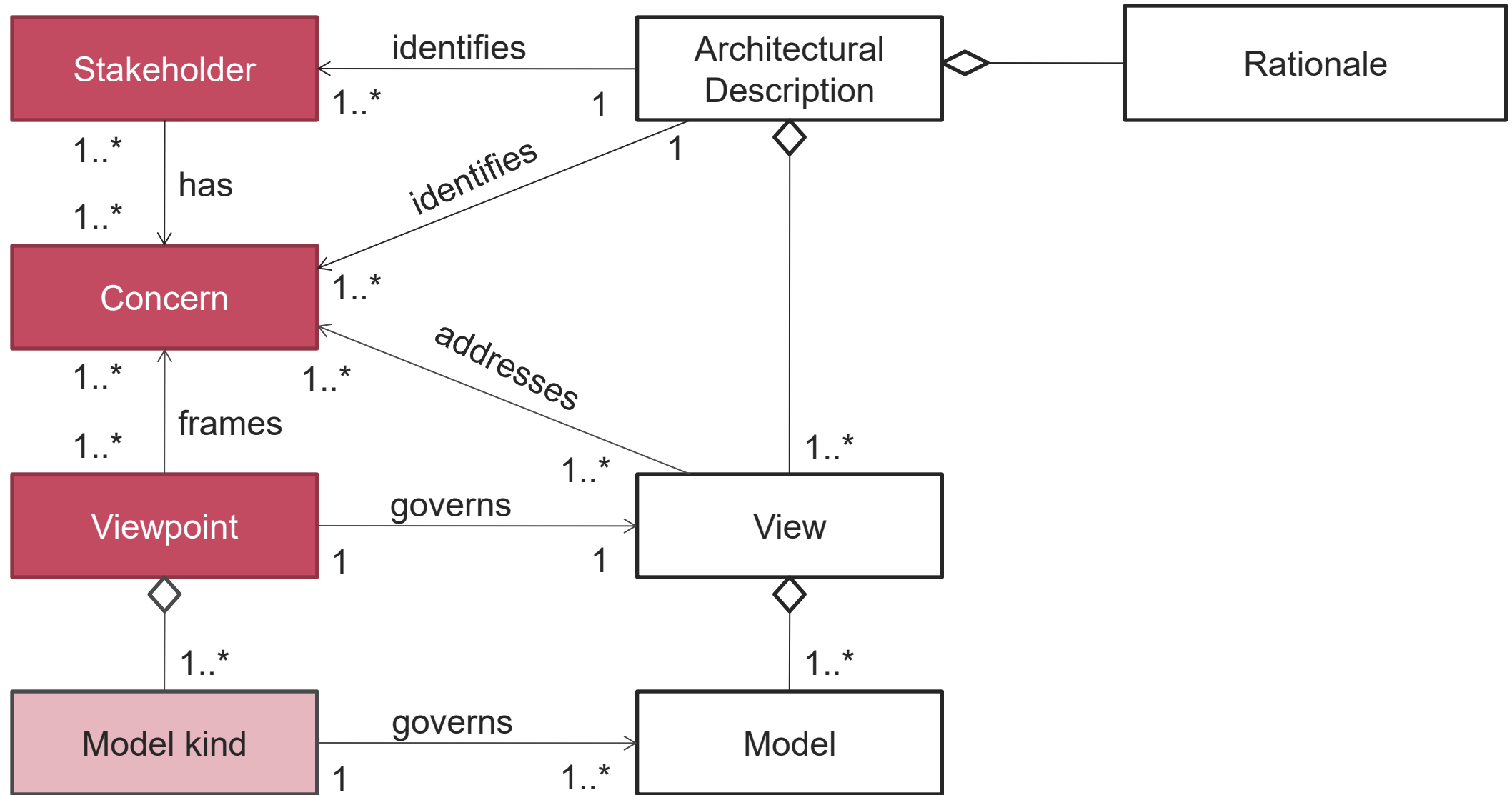
Architectural framework

*Which **views** do I have to provide?*

*Which **viewpoints** are suitable for which **stakeholders** and which **concerns** do they highlight?*

2. Architectural frameworks

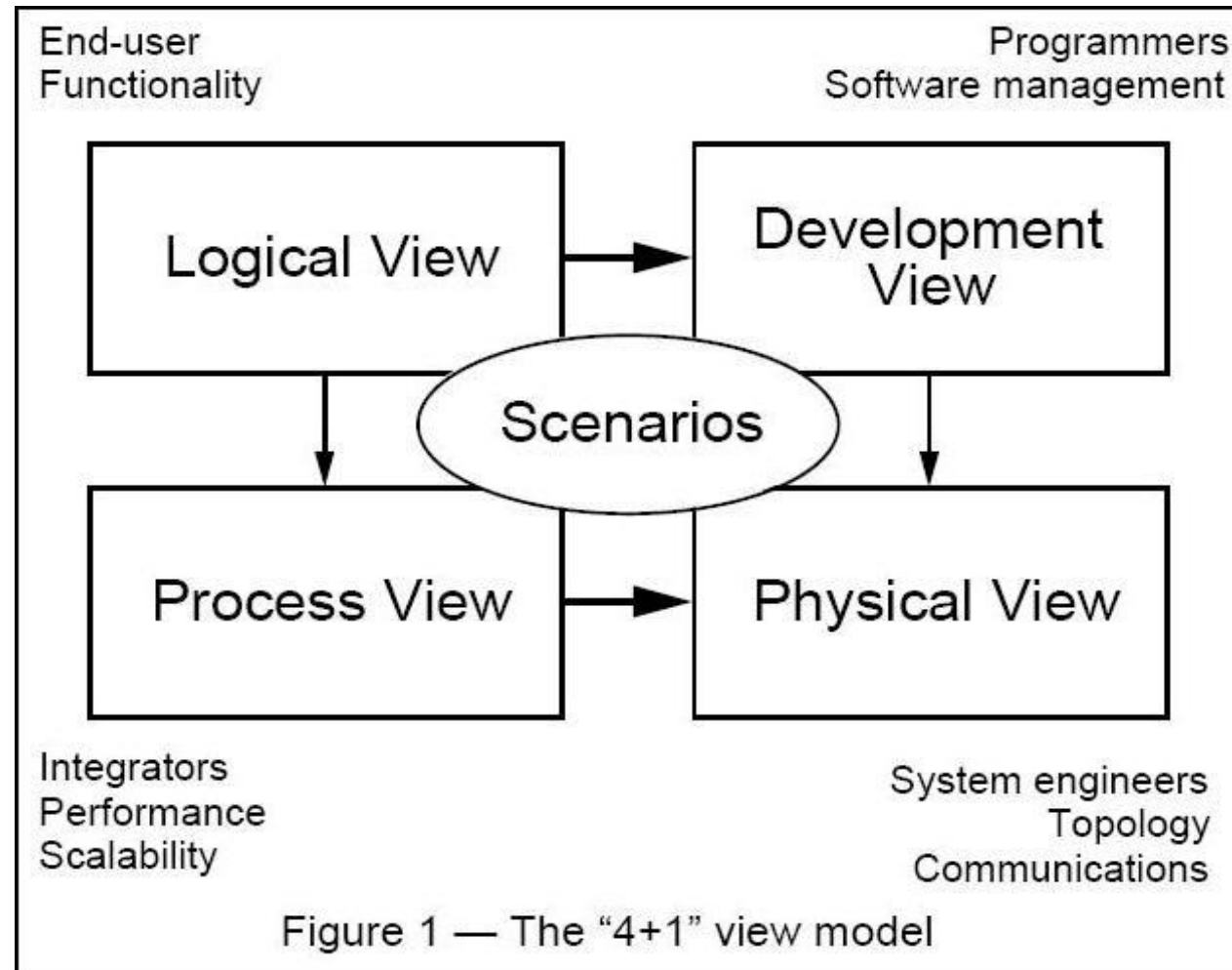
Architectural frameworks



ISO 42010

Architectural viewpoints

- › Different frameworks propose different viewpoints



Kruchten [95]

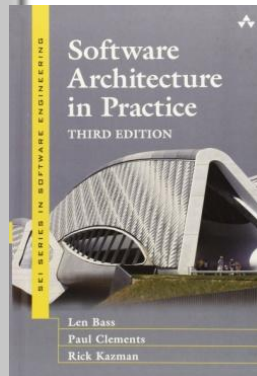
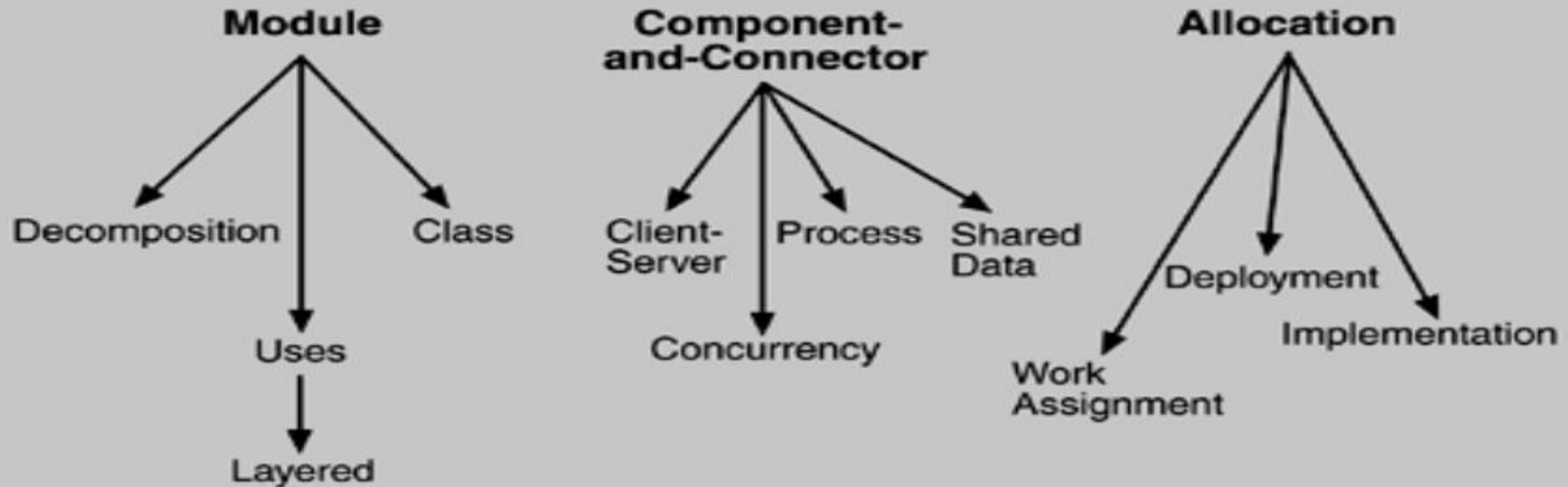
Architectural viewpoints

- › Different frameworks propose different viewpoints

	DATA <i>What</i>	FUNCTION <i>How</i>	NETWORK <i>Where</i>	PEOPLE <i>Who</i>	TIME <i>When</i>	MOTIVATION <i>Why</i>
Objective/Scope (contextual) <i>Role: Planner</i>	List of things important in the business	List of Business Processes	List of Business Locations	List of important Organizations	List of Events	List of Business Goal & Strategies
Enterprise Model (conceptual) <i>Role: Owner</i>	Conceptual Data/ Object Model	Business Process Model	Business Logistics System	Work Flow Model	Master Schedule	Business Plan
System Model (logical) <i>Role: Designer</i>	Logical Data Model	System Architecture Model	Distributed Systems Architecture	Human Interface Architecture	Processing Structure	Business Rule Model
Technology Model (physical) <i>Role: Builder</i>	Physical Data/Class Model	Technology Design Model	Technology Architecture	Presentation Architecture	Control Structure	Rule Design
Detailed Representation (out of context) <i>Role: Programmer</i>	Data Definition	Program	Network Architecture	Security Architecture	Timing Definition	Rule Speculation
Functioning Enterprise <i>Role: User</i>	Usable Data	Working Function	Usable Network	Functioning Organization	Implemented Schedule	Working Strategy

Architectural viewpoints

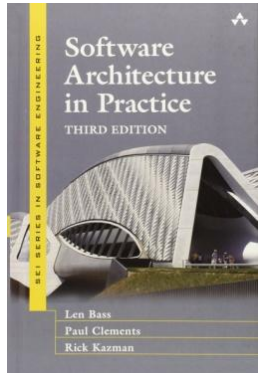
- › Different frameworks propose different viewpoints



Architectural viewpoints

KEY

s some details
d detailed information
o overview information
x anything



	Module				C&C	Allocation	
Stakeholder	Decomposition	Uses	Class	Layer	Various	Deployment	Implementation
Project Manager	s	s		s		d	
Member of Development Team	d	d	d	d	d	s	s
Testers and Integrators		d	d		s	s	s
Maintainers	d	d	d	d	d	s	
Product Line Application Builder		d	s	o	s	s	s
Customer					s	o	
End User					s	s	
Analyst	d	d	s	d	s	d	
Infrastructure Support	s	s		s		s	d
New Stakeholder	x	x	x	x	x	x	x
Current and Future Architect	d	d	d	d	d	d	s

Three 'classes of' architectural viewpoints

» **Component-and-connector viewpoints** (runtime)

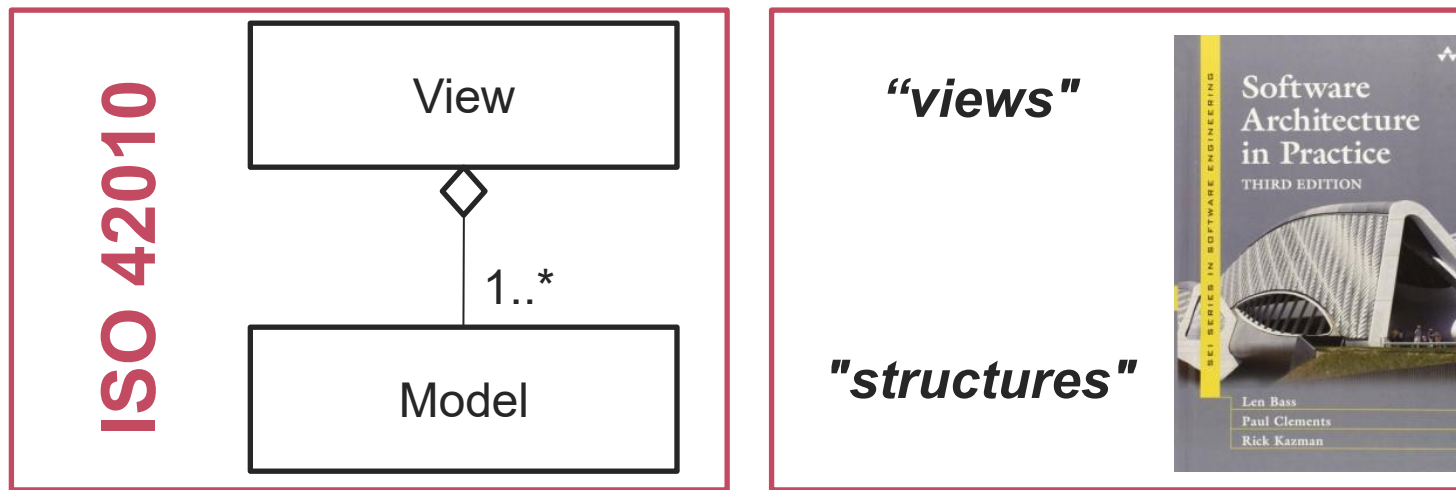
»» How the software is structured as a set of **elements that have runtime behavior (components)** and their **interactions (connectors)**

» **Module viewpoints** (construction, design)

»» How the software is structured as a set of **implemented units having responsibilities**

» **Allocation viewpoints** ("mapping" x to y)

»» How the software **relates to non-software structures in the environment**



C&C viewpoints in general

Components and connectors

› Elements

- › A **component** is an element with **runtime presence** (processes, clients, servers, and data stores)
 - ›› Components can have *ports*, i.e., points of (potential) interaction at runtime
- › A **connector** is an interaction means between the components (communication, information flows)
 - ›› the visibility between two parts
 - ›› a communication means

› Relations

- › Attachment of **connectors** to components to form a graph
 - ›› Who talks to who, and how; control flow/data flow/ ...

Module viewpoints in general

› Elements

- › A **module** is an **implementation unit** that provides a coherent set of responsibilities.
 - ›› E.g., a class, a collection of classes, a layer, an aspect, or any decomposition of the implementation unit.
- › **Structural elements** and their relationships

› Relations

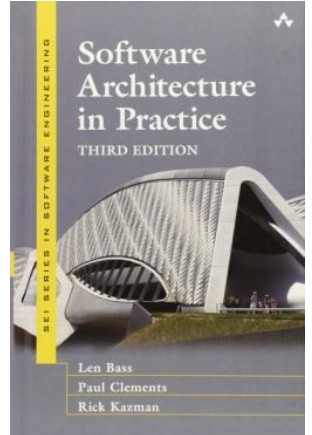
- › Is part of → decomposition and aggregation
- › Depends on → dependencies between modules
- › Is a → generalization/specialization

Allocation viewpoints in general

- › Elements
 - › Software element and **environmental** element
- › Relations
 - › Allocated-to (distribution of software)
- › E.g. work assignment view (teams/developers), implementation (files), deployment view (machines, nodes)

In this course...

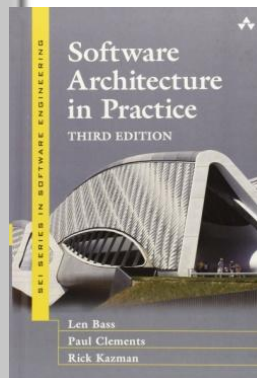
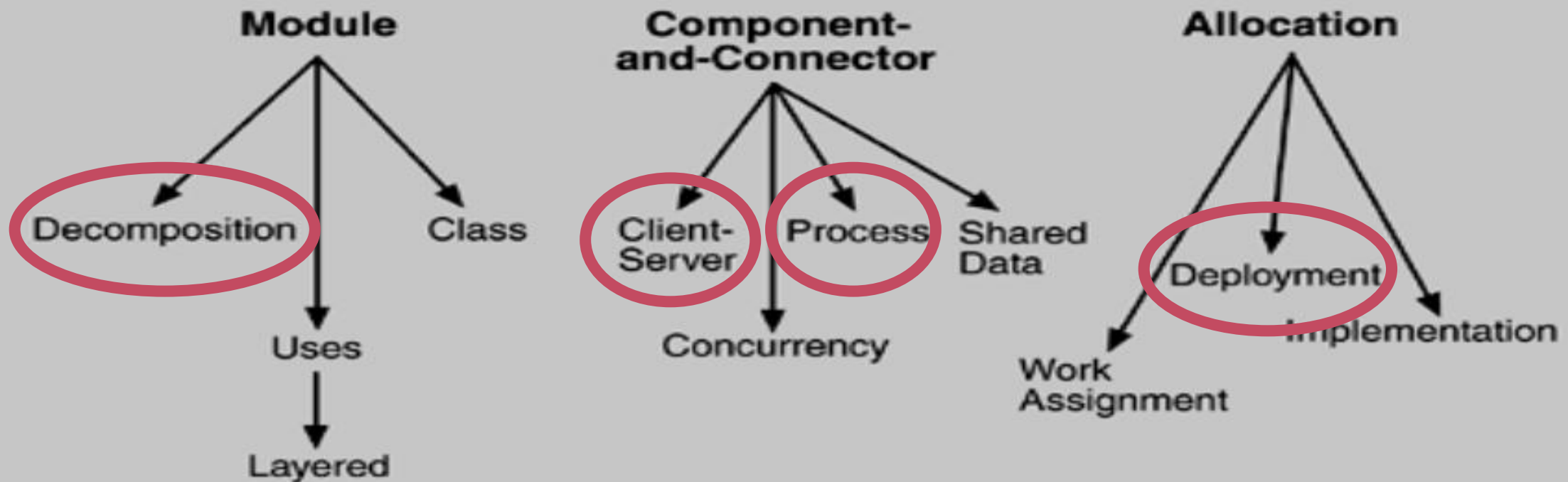
- › Terminology aligned to **ISO 42010**
- › **Architecture framework:** from **textbook**

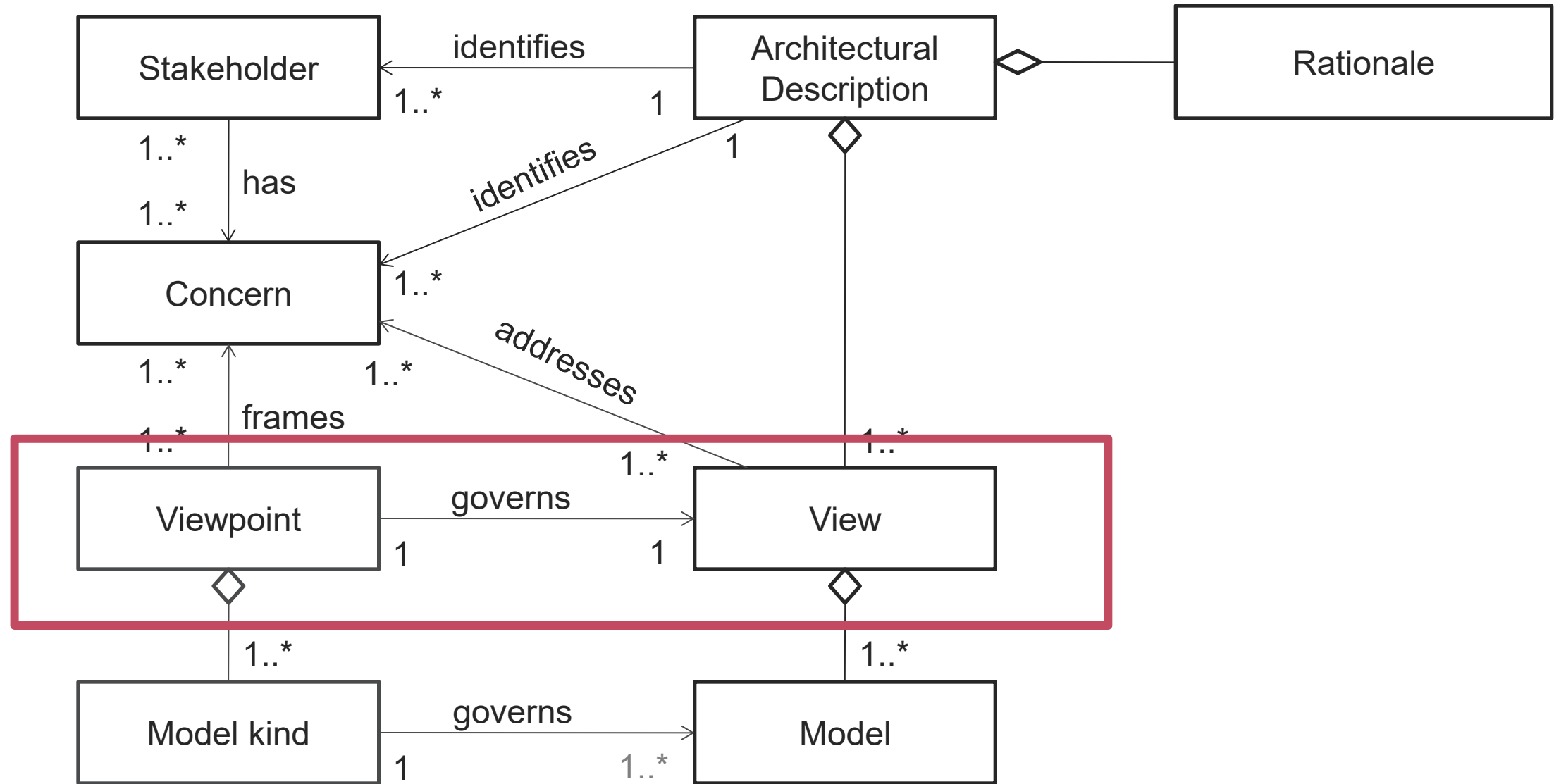


3. **Four** specific architectural viewpoints
used in this course

In this course...

› Four specific architectural viewpoints





1. Client-server viewpoint (type: C&C)

Key abstraction: **components**

Purpose

- › **Run-time, software entities (*components*)** interact in a **client-server relation** (connectors: ***A uses/interacts with B***) – interaction is a ***connector***
- › To answer the following questions
 - ›› What are the main data stores?
 - ›› What is the flow of control through the system?
 - ›› How does data progress through a system as it executes?
 - ›› What parts of the system run in parallel?
 - ›› How can the system's structure change as it executes?
- › **Concerns:** Performance, reliability, functionality, security, ...

2. Process view (type: C&C)

Key abstraction: **components**

Purpose

- › *Refining* the connectors (how does **A** *use/interact with* **B**?)
 - » Process, protocol, state-based behavior
- › Timing
 - » Sequential messages
 - » Synchronous/asynchronous messages
 - » Conditional behavior
 - » Loops
 - » Exceptions
 - » Relationships between inputs and outputs
- › **Concerns:** Performance, availability, functionality, security, ...

3. Decomposition view (type: module)

Key abstraction: **modules**

Purpose

› Communication

- ›› The various levels of granularity of the **module** decomposition provide a **top-down presentation** of the system's responsibilities

› Construction

- ›› Design structure
- ›› Blueprint for the source code (construction point of view)
- ›› Reuse of libraries
- ›› Modularization/modularity
- ›› (De)coupling and (de)compose

› **Concerns:** Modifiability, extensibility, interoperability, reusability, etc

4. Deployment view (type: allocation)

Key abstraction: **nodes**

Purpose

- › Environmental elements
 - › Where software components reside **at run-time**; nodes (=machines), clusters (virtualized or not)
 - › Hardware (computers, processors, devices, peripherals), application servers, etc.
- › Identify the **distribution** architecture
 - › Do you intend to take a fat-client approach where the **business logic is contained in a desktop** application or a thin-client approach where business logic is deployed to an application server?
 - › Will your application have two **tiers**, three tiers, or more?
 - › What **HW and OS** are to be deployed?
 - › How will the various nodes be connected (perhaps via **RMI** and a **message bus**), protocols, physical channels, etc
 - › **Replication** and multiple component instances
- › Important for Installation/configuration
- › **Concerns:** **Availability, performance, portability, data consistency, scalability, reliability, ...**

Recap – in this course

- › Units of development/construction: **modules**
 - ›› Classes, packages, libraries, ...
 - ››› used in *decomposition view (module)*
- › Units of “the running system”: **components**
 - ›› Processes, threads, services, workers, ...
 - ››› Used in *client-server* and in *process view (component&connector)*
- › Units of deployment/installation: **nodes**
 - ›› Machines, containers, clusters, devices, (physical) terminals, phones, ...
 - ››› Used in *deployment view (allocation)*
- › Units = *abstractions*

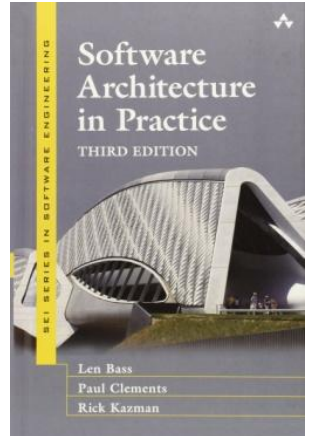
In this course...

› Terminology aligned to **ISO 42010**

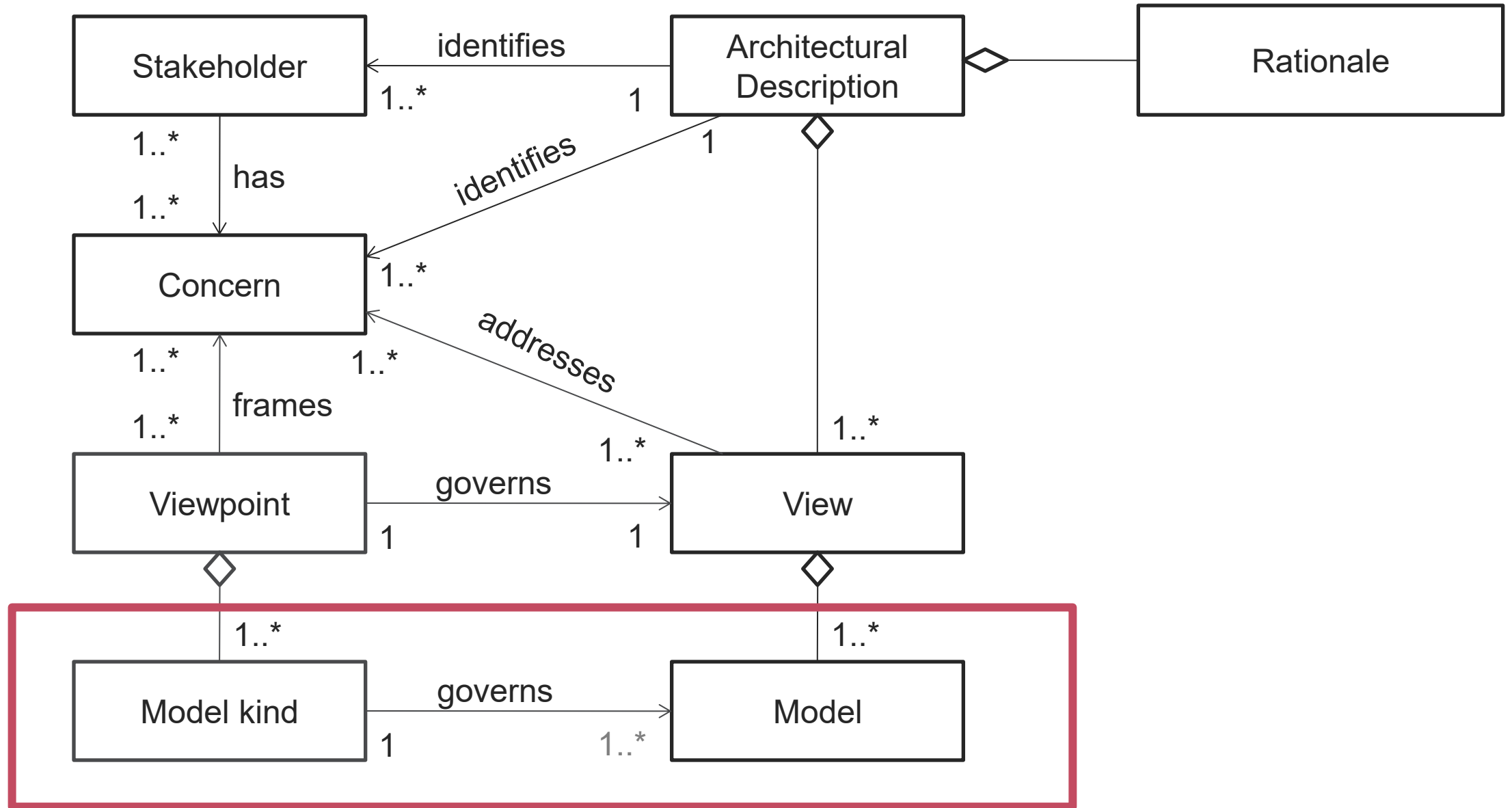
› **Architecture framework:**

Our selection of architectural viewpoints from textbook

1. Client-server (C&C)
2. Decomposition (Module),
3. Process (C&C) and
4. Deployment (Allocation)

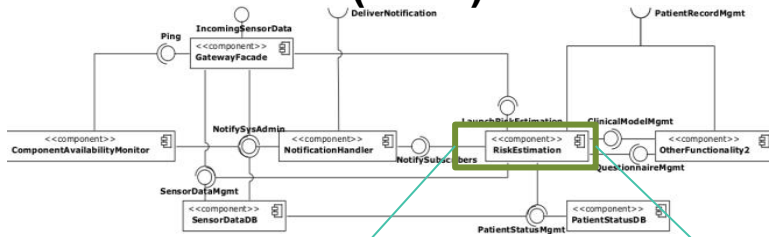


4. Model kinds



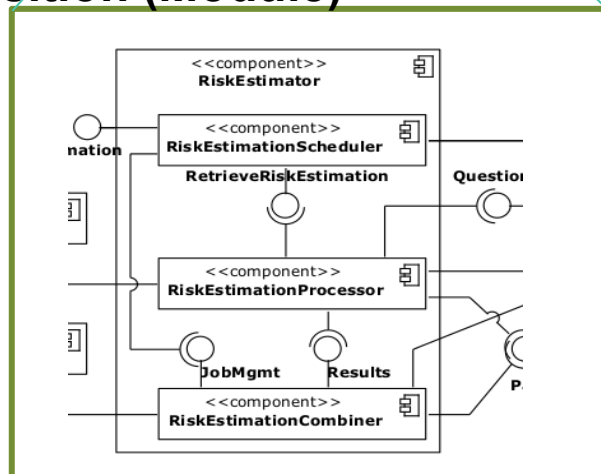
Four architectural viewpoints

Client-Server (C&C)



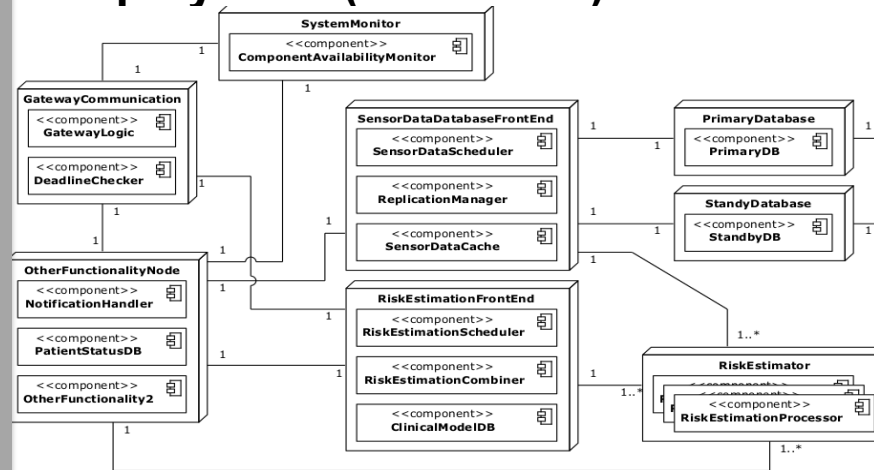
UML Component diagram

Decomposition (Module)

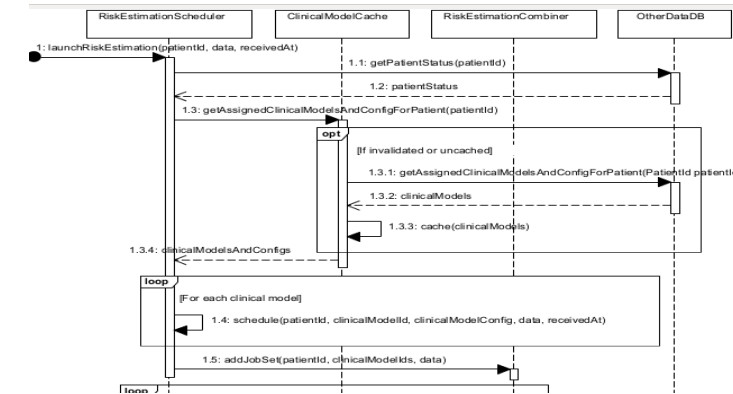


UML Component diagram

Deployment (Allocation)



UML Deployment diagram



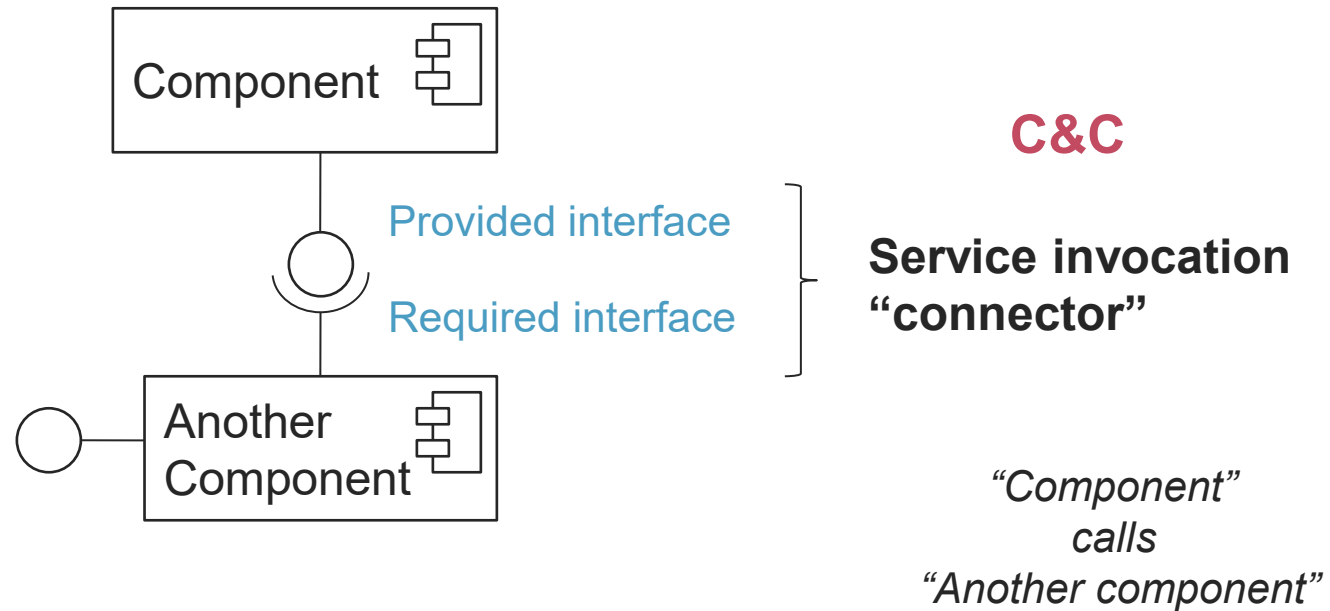
Process view (C&C)

UML Sequence diagram

Black: Viewpoints
BLUE: UML diagram types

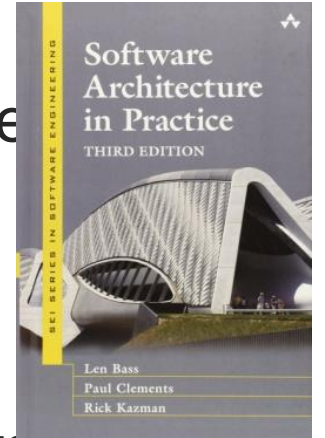
1. Client-server view (C&C)

Component and connectors in UML



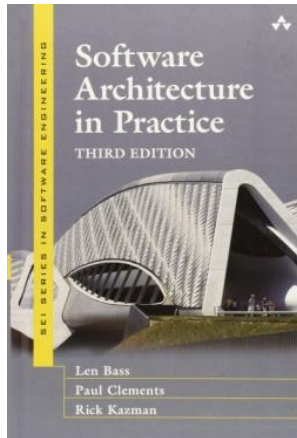
Components & connectors in the client-server view

- › **Component:** Run-time software element, with its own behaviour, state
 - ›› Processes, threads, services, workers, ...
- › **Connector:** governs the run-time interaction between two components
 - ›› In our notation: matching a **provided interface** with a **required interface**
 - ›› **Provided:** offered services/API endpoint
 - ›› **Required:** services provided by other components
 - ››› Availability of resources (e.g. config file, access to storage, ...)
 - ››› Run-time (e.g. execution environment)



Interfaces

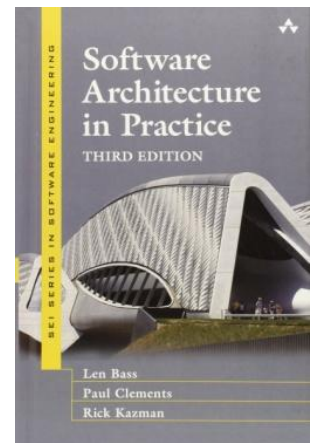
- › Components can be seen as a black boxes (*encapsulation*)
- › Interact by means of interfaces that **partition** details about an element into public and private parts
- › **Ingredients of an interface**
 - › Name of the services
 - › Parameters
 - › Contract (pre/post conditions)
- › Enables **reuse** and **evolution**



Connector

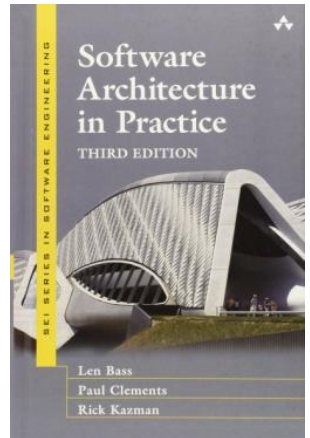
- › A software connector is an architectural element tasked with **effecting and regulating interactions** among components
- › Roles
 - › **Communication**: support transmission of data among components
 - › **Coordination**: passing the thread of execution
 - › **Conversion**: adaptation of the interaction

*"Having divided to conquer,
we must reunite to rule." —
Michael Jackson (1990)*



Connector types

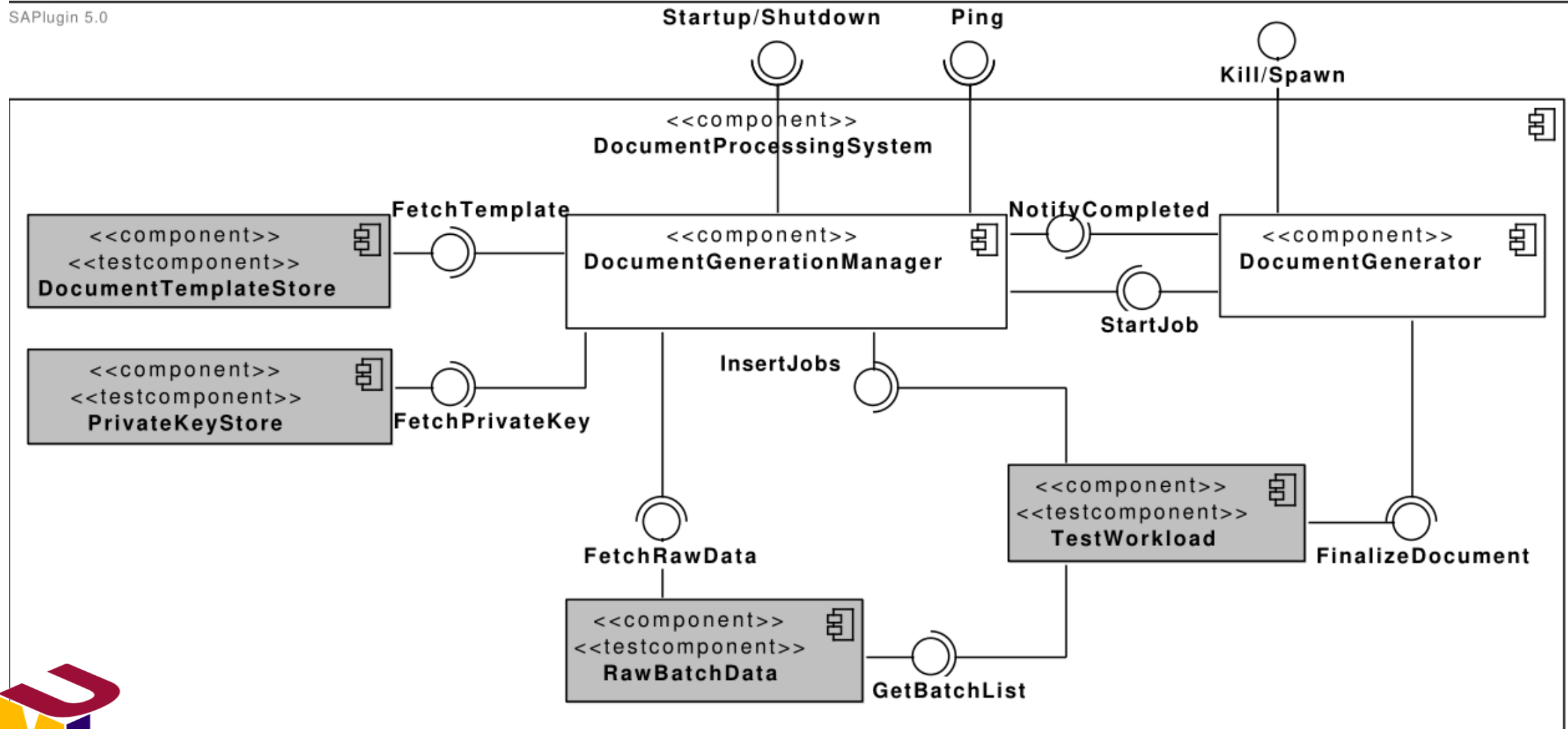
- › Sync/async **service invocation** (communication and coordination)
- › Async **event notification** (pub/sub)
- › Distributor (location-transparent routing)
- › Data access (SQL and query mechanisms)
- › Stream (TCP/UDP)
- › Adapter (wrapper and glue code)
- › Arbitrator (semaphores)
- › ...



Client-server view (C&C) in UML

Component Diagram

SAPPlugin 5.0



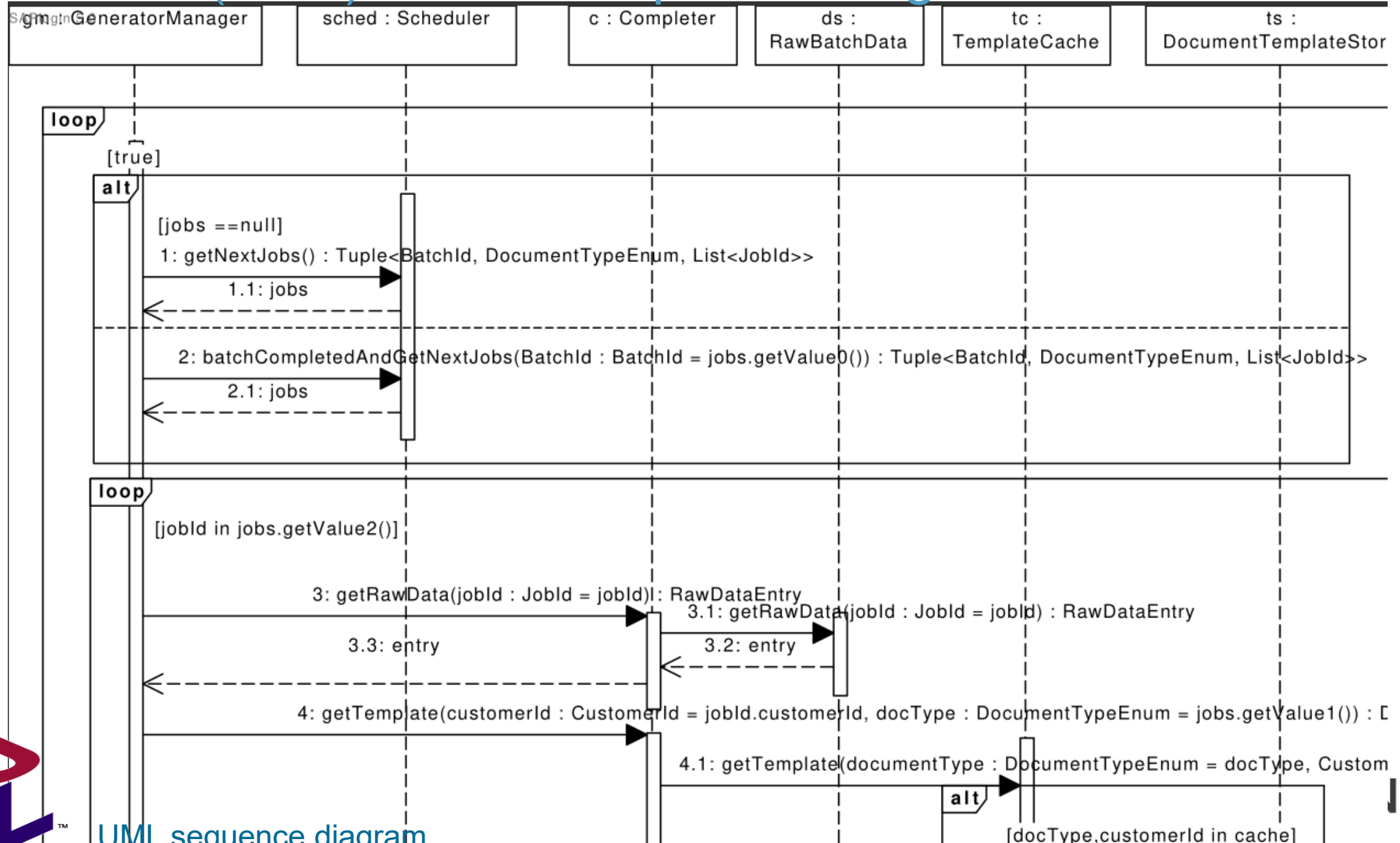
2. Process view (C&C)

Process view (C&C)

Purpose

- › Further *refinement* of the connectors
 - ›› Process, protocol, state-based behavior
- › Timing
 - ›› Sequential messages, causality in behavior,
 - ›› Synchronous/asynchronous messages
 - ›› Conditional behavior
 - ›› Loops
 - ›› Exceptions

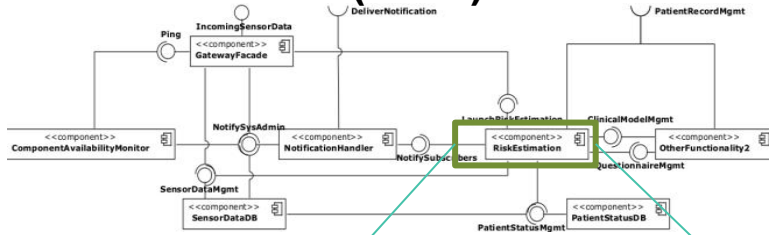
Process view (C&C) in UML Sequence diagram



3. Decomposition view (Module)

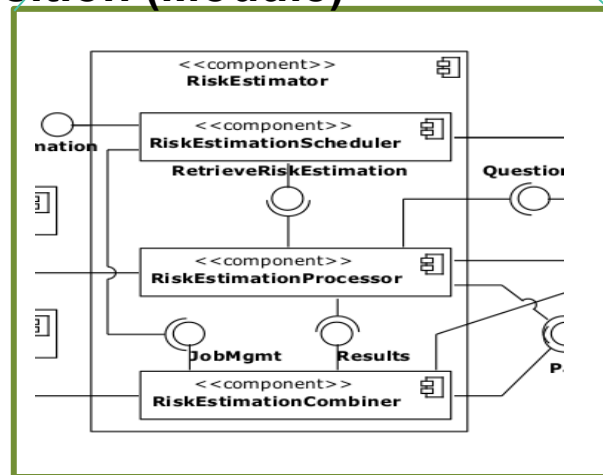
Module viewpoint

Client-Server (C&C)



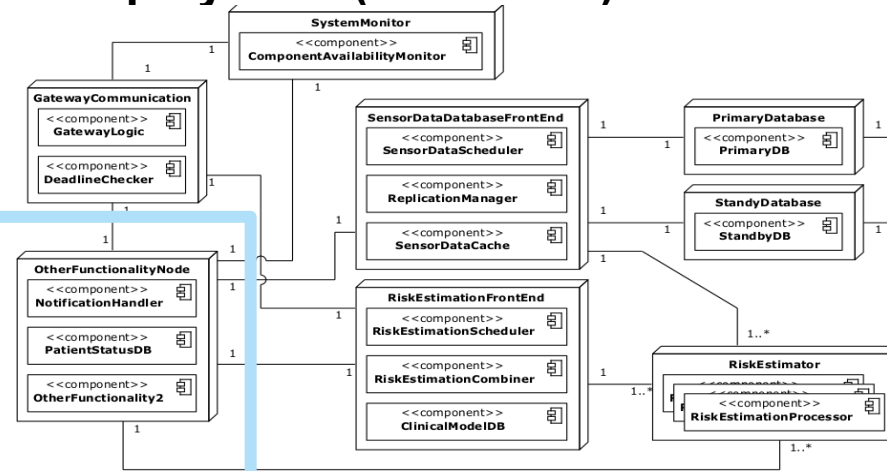
UML Component diagram

Decomposition (Module)

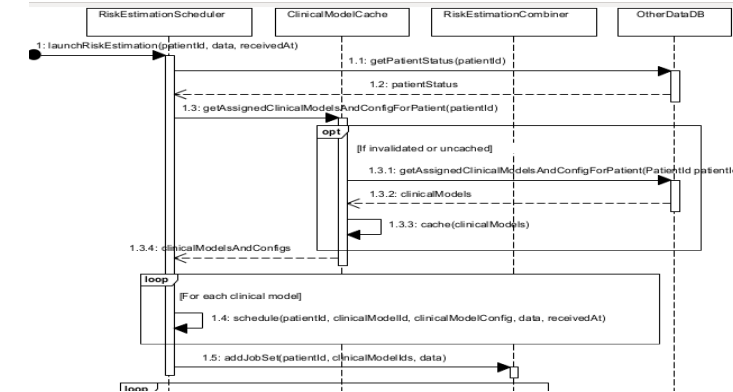


UML Component diagram

Deployment (Allocation)



UML Deployment diagram



Process view (C&C)

UML Sequence diagram

Black: Viewpoints

BLUE: UML diagram types

Decomposition view (module)

Purpose

› Roles and responsibilities

- ›› The various levels of granularity of the module decomposition provide a **top-down presentation** of the system's responsibilities

› Construction/structure

- ›› Blueprint for the source code
- ›› Reuse and dependency on libraries
- ›› How to fit in extensions/plugins
- ›› Modularity

Decomposition view (module)

in UML

- › Different types of diagrams can be used
 - › Composite structure diagrams
 - › Class diagrams
- › In this course, we use **component diagrams** as well
- › But **remember well** the distinction between: **module** and **component**
 - › Same diagram type, same element type, yet **different viewpoint and different semantics**



Course convention: use the `<<module>>` stereotype to make the distinction

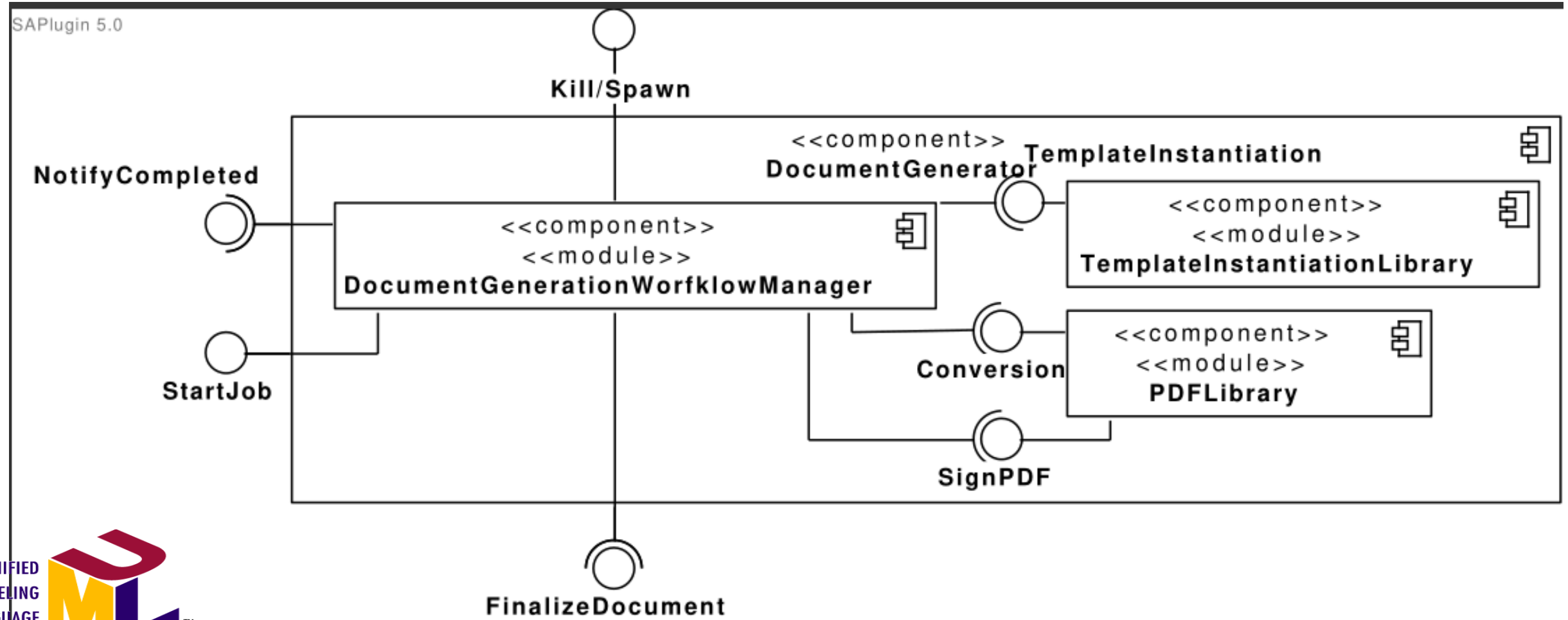
Decomposition view (module)

Module vs. component?

- › Module = **implementation** unit
 - › E.g., a project, package, class file, layer, library, ...
- › Component = **run-time** unit
 - › E.g., a process, object, service, active API endpoint, database instance, thread, ...
- › In general, there is **a complex many-to-many relationship**
- › **In this course**, we adopt a different interpretation of a `<<component>>` and this depends on the view:
 - › Client-server view: component / run-time unit
 - › Decomposition view: module / implementation unit
 - › Process view: component / run-time unit
 - › Deployment view: component / run-time unit

Decomposition view (module)

in UML: component diagram

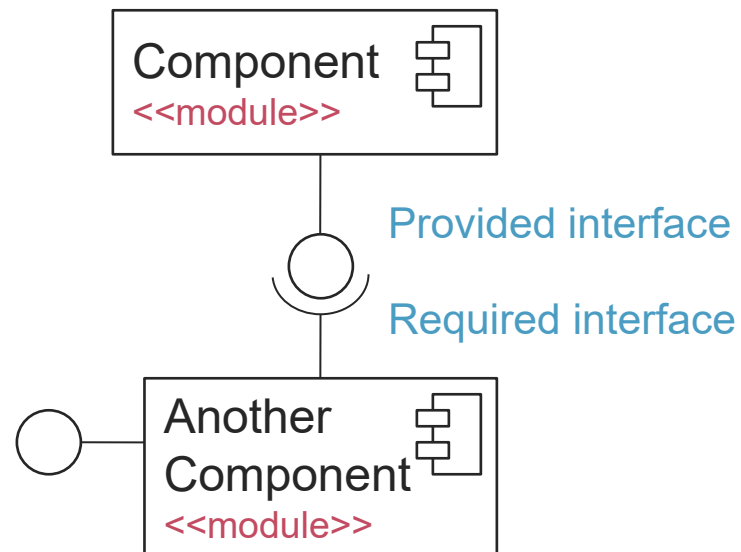


UML component diagram

Not “separation in run-time entities”, but design structure

Component and connectors in UML

› Simplest form/notation



C&C

**Service invocation
"connector"**

*"Component"
calls
"Another component"*

Module

Code dependency

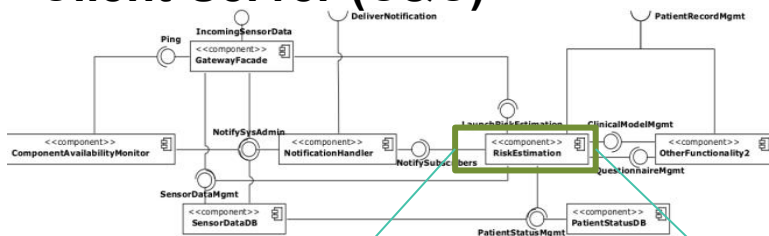
*The implementation
of "Component"
depends on/is
built upon
"Another component"*

4. Deployment view (Allocation)

Allocation viewpoint

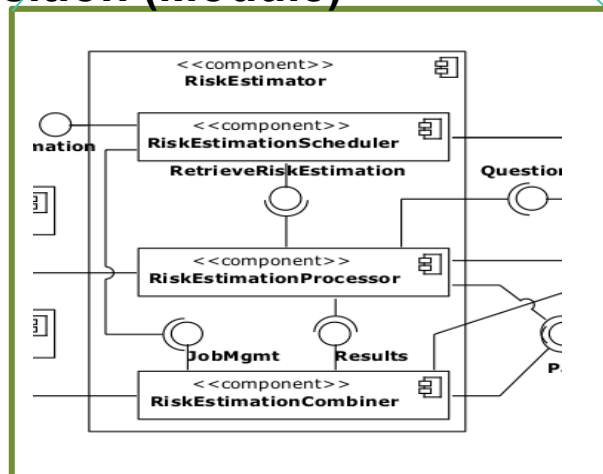
LOGICAL

Client-Server (C&C)



UML Component diagram

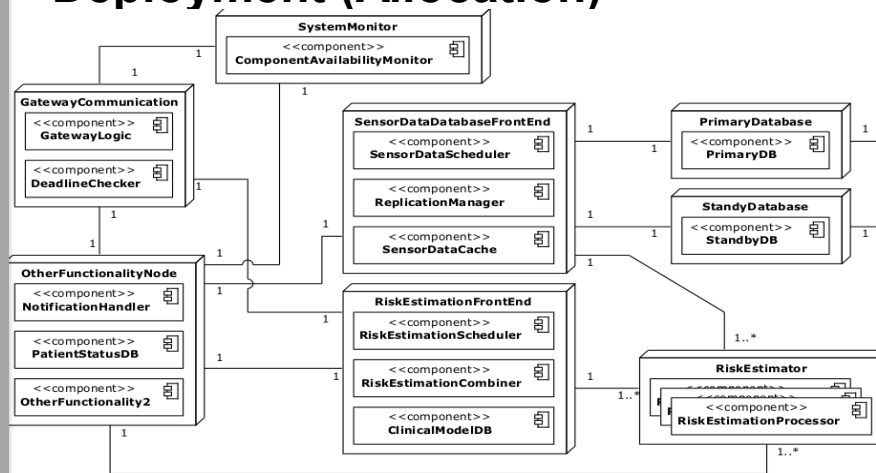
Decomposition (Module)



UML Component diagram
(UML Composite structure diagram,
UML Class diagram)

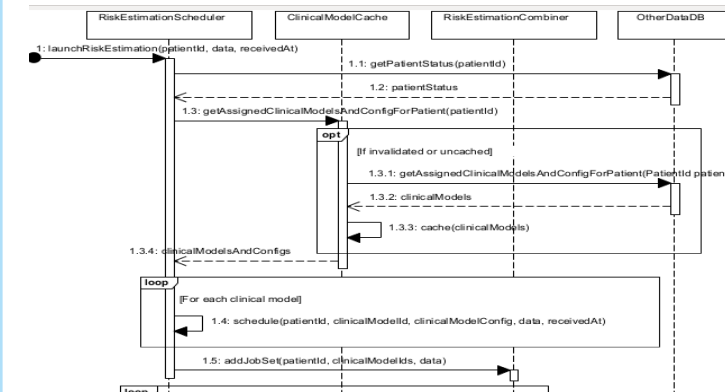
PHYSICAL

Deployment (Allocation)



UML Deployment diagram

PROCESS



Process view (C&C)

UML Sequence diagram
UML Communication diagram

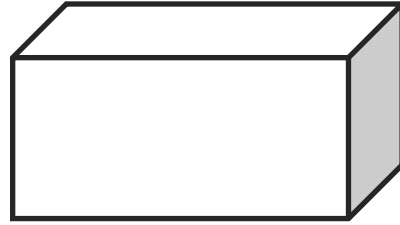
GREEN: Kruchten terminology

Black: Textbook viewpoints

BLUE: UML diagram types

Deployment view (allocation)

in UML



› Nodes

- › Computational resources of a system, such as personal computers, sensors, printing devices, or servers
- › Can have stereotypes (devices, execution environment, ...)

› Nodes **may have sub-nodes**, which appear as nested boxes (*not very common*)

› A single node may conceptually represent multiple physical nodes

- › E.g. a cluster of database servers
- › → **avoid this in your project**

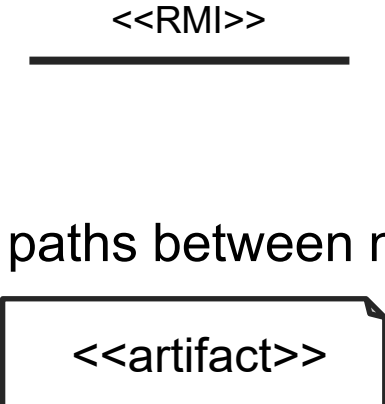
Deployment view (allocation)

in UML

› Communication links

- ›› Nodes are interconnected, to represents communication paths between nodes

<<RMI>>



› Artifacts

- ›› A product of the software development process
- ›› Including config files, executables, user manuals, etc.

Deployment view (allocation)

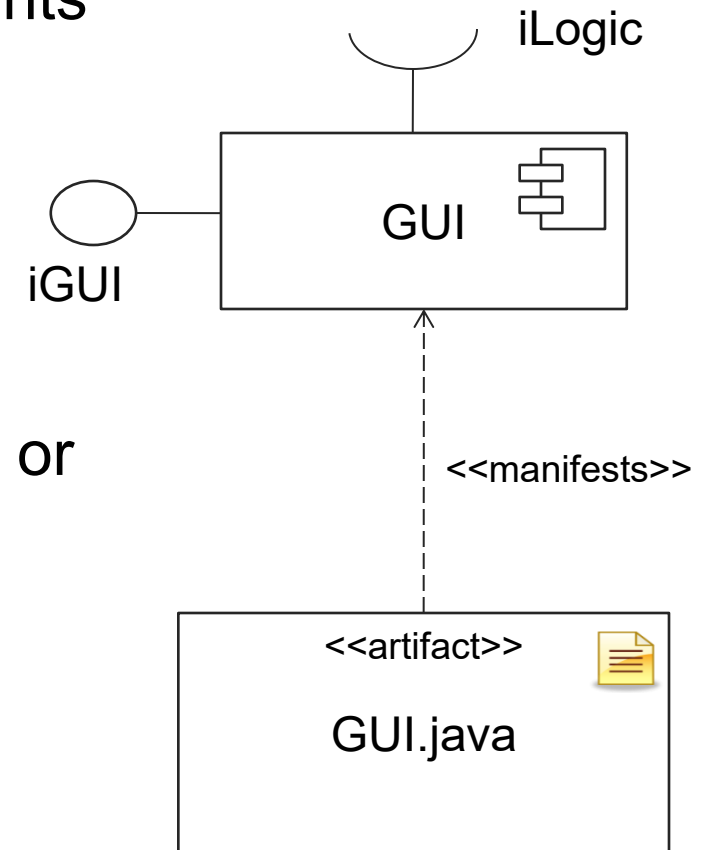
in UML: artifacts and components

› An **artifact** <<manifest>> one or more model elements

›› This model element often is a component

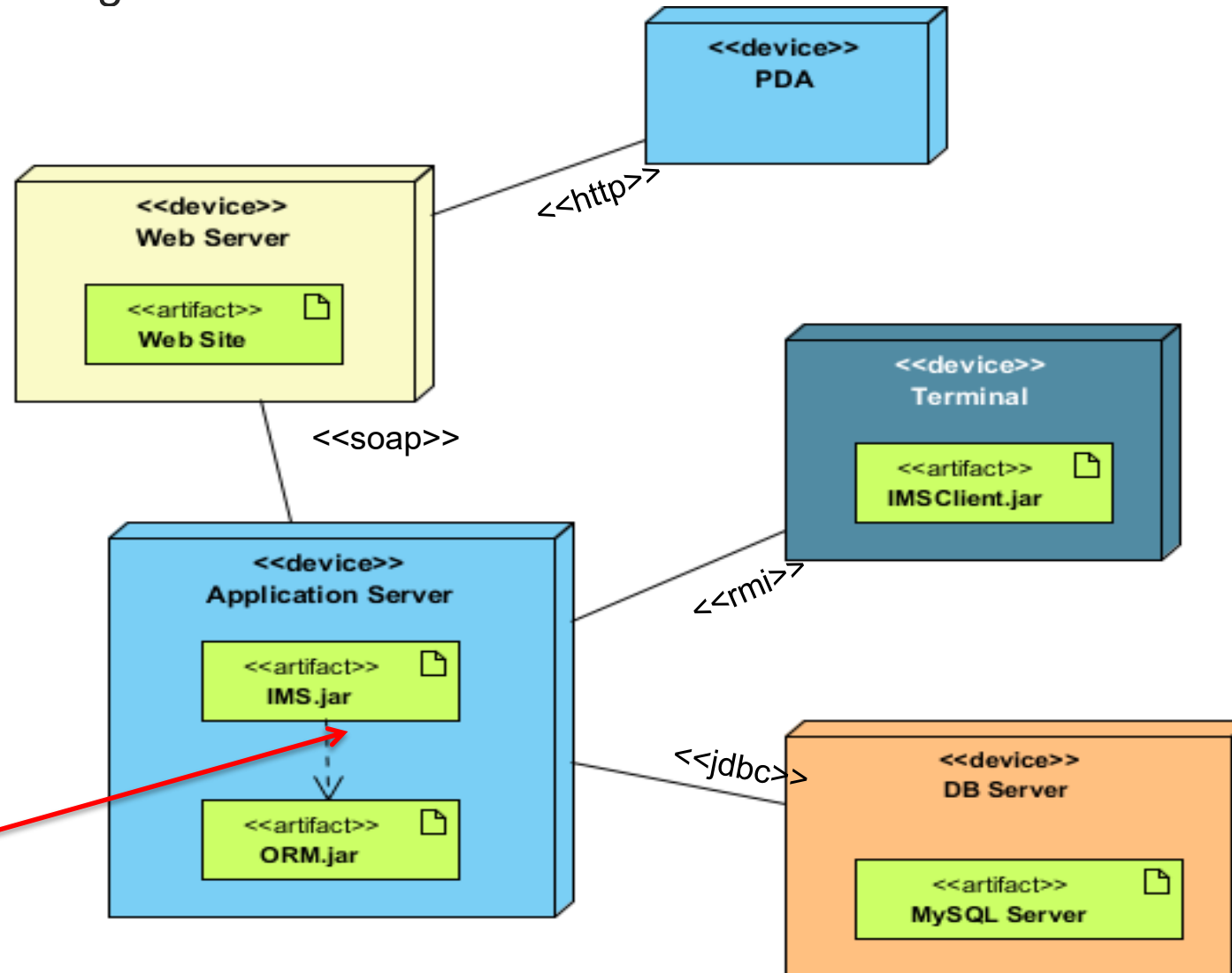
›

A “manifestation” is the concrete incarnation of one or more model elements



Deployment view (allocation)

in UML: deployment diagram



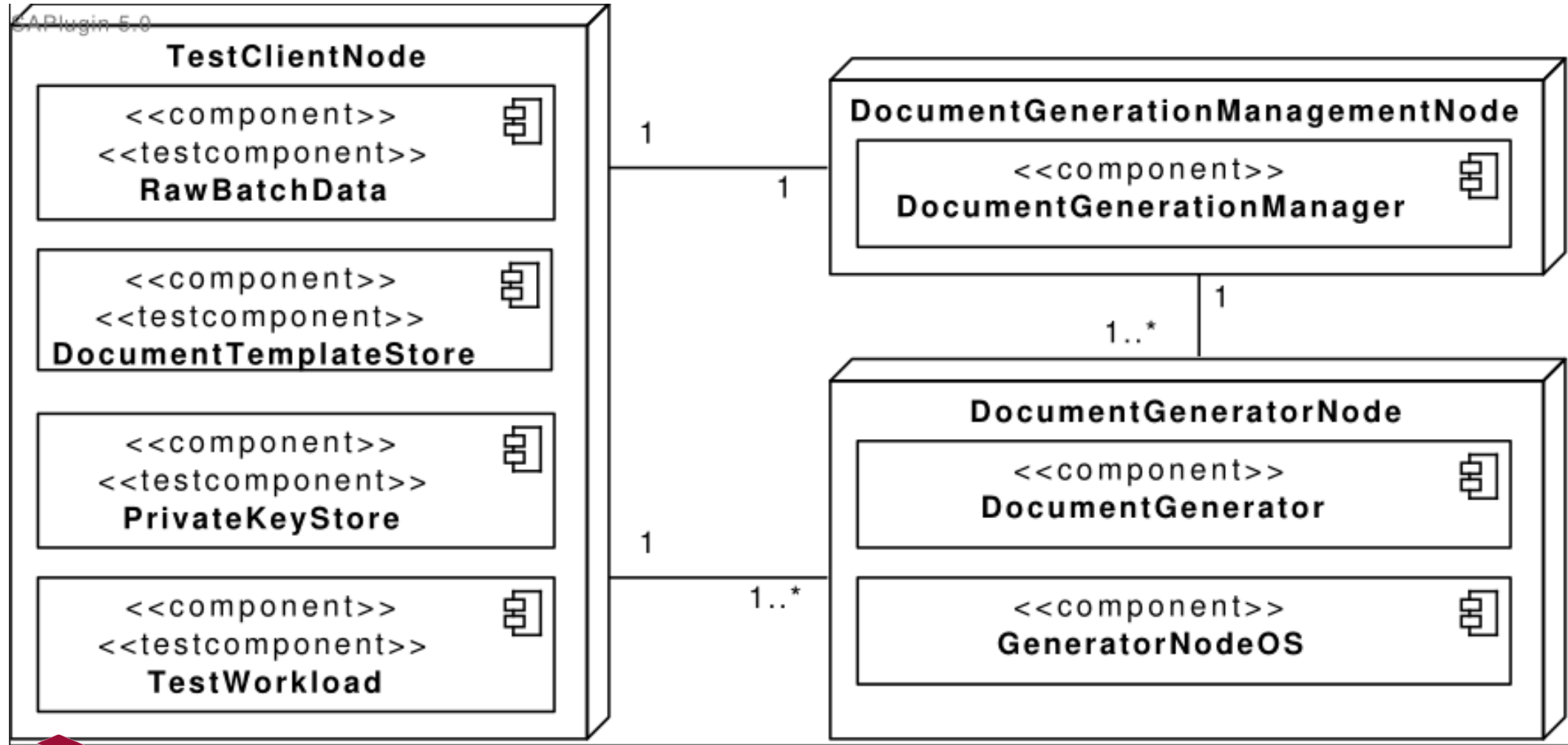
Dependencies
(e.g libraries)

In this course

- › We **don't distinguish** between artifacts and components
 - › i.e., we assume a 1-1 mapping
- › So you may deploy your components directly onto nodes
 - › Strictly speaking not allowed in UML
- › You may also deploy a component multiple times on one node
 - › Meaning: multiple run-time instances (e.g., processes, threads, ...) of the same component type

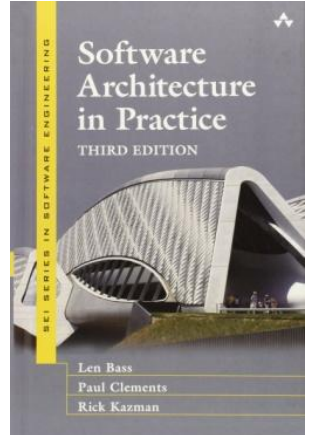
Deployment view (allocation)

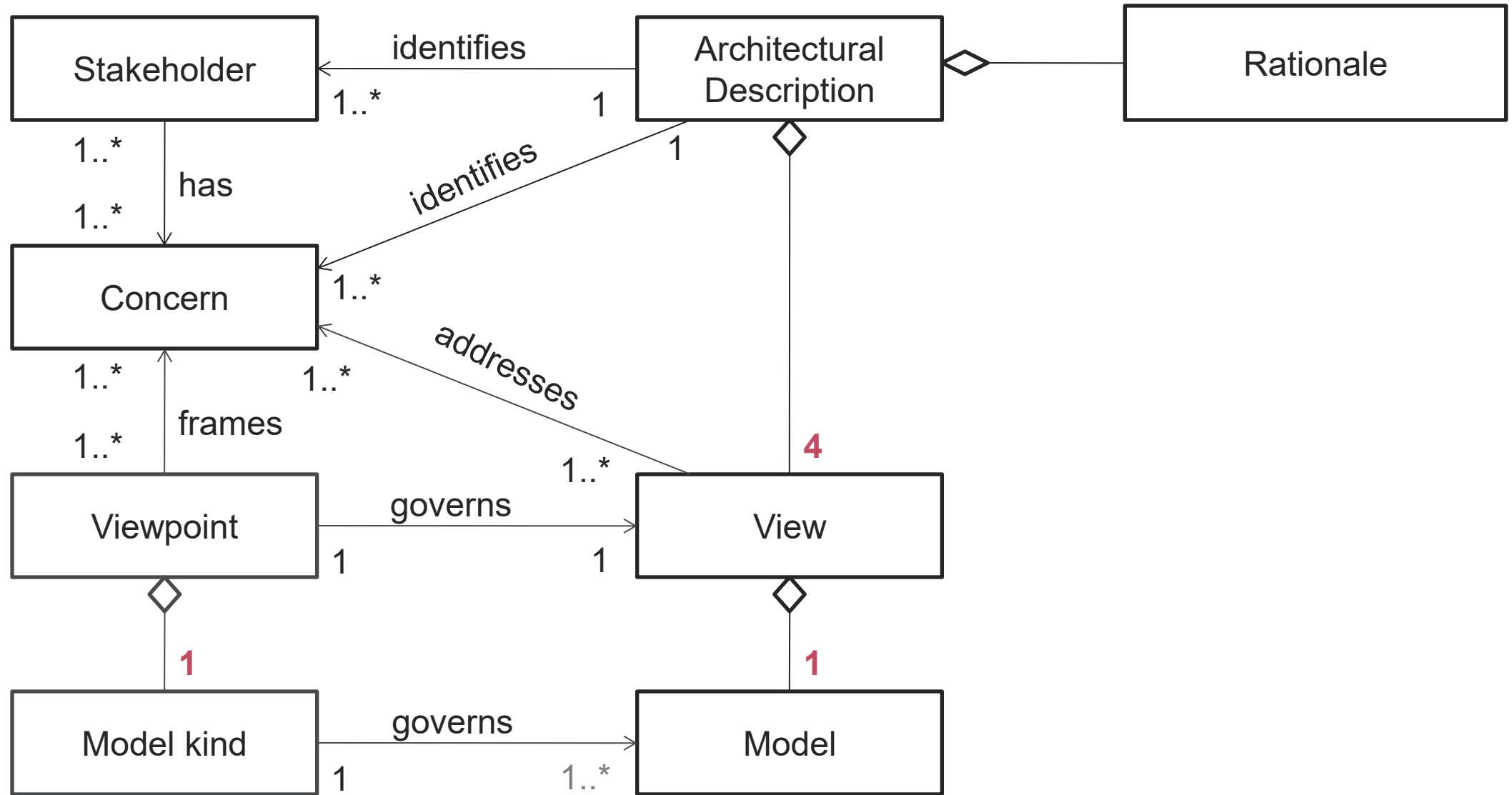
in UML: deployment diagram from Patient Monitoring System



In this course...

- › Terminology aligned to **ISO 42010**
- › **Architecture framework:**
Our selection of architectural viewpoints from **textbook**
 1. **Client-server (C&C)**
 - Model kind: [UML Component diagrams](#)
 2. **Decomposition (Module)**
 - Model kind: [UML Component diagrams](#)
 3. **Process (C&C)**
 - Model kind: [UML Sequence diagrams](#)
 4. **Deployment (Allocation)**
 - Model Kind: [UML Deployment diagrams](#)





Sufficiently
rich/expressive for
discussing the
most relevant
concerns

This week's definition

- › Software Architecture (description)
 - ›› a collection of *views*, [addressing *stakeholder concerns* according to well-defined *viewpoints*, further concretized in specific *models* (e.g. diagrams)], augmented with *rationale*

Definitions of SA, so far

- › Lecture 1: “*the bridge between requirements and design*”
- › Lecture 2: “*meeting or maximizing utility*”
- › Lecture 3: “*{elements, form, rationale}*” | “*the set of principal design decisions*” | “*the fundamental organization of a system, embodied in its components, their relationships [...], and the principles guiding its design and evolution*”
- › Lecture 4: SA description = a collection of *views*, [addressing *stakeholder concerns* according to well-defined *viewpoints*, further concretized in specific *models* (e.g. diagrams)], augmented with *rationale*